

```
[fedora@h-mstr ~]$ cd /usr/share/elasticsearch/
[fedora@h-mstr elasticsearch]$ sudo bin/plugin -install \
    elasticsearch/elasticsearch-mapper-attachments/2.4.1
[fedora@h-mstr elasticsearch]$ sudo systemctl start
elasticsearch
```

Repeat the steps on *h-slv1* and *h-slv2*.

You will need to create an index of the type of attachment before you can start indexing the documents. The following Curl script will create the index 'documents':

```
$ curl -XPOST h-mstr:9200/documents -d '{
  "mappings" : {
    "attachment" : {
      "_source": {"excludes" : ["file"] },
      "properties" : {
        "file" : {
          "type" : "attachment",
          "fields" : {
            "title" : { "store" : "yes" },
            "file" : { "term_vector":"with_
positions_offsets", "store":"no" } } } } }'
```

Since you are storing and accessing the files from HBase, there is no advantage in storing the documents in the index as well; hence, the directive to exclude 'file' from being stored in the source.

Extend 'store_files_in_hbase.py' to include indexing of the document as well as using <https://gist.github.com/stevehanson/7462063/> as a reference:

```
TMP_FILE_NAME = 'tmp.json'
def create_encoded_temp_file(fname, filecontent):
    import json
    file64 = filecontent.encode("base64")
    f = open(TMP_FILE_NAME, 'w')
    data = { 'file': file64, 'title': fname }
    json.dump(data, f) # dump json to tmp file
    f.close()

def process_file(table, p, f):
    filename = '.'.join([p, f])
    filecontent = open(filename, 'rb').read()
    print("Processing %s %s Len %d"%(p, f, len(filecontent)))
    table.put(filename, {'content':data':filecontent'})
    # Index the contents
    create_encoded_temp_file(filename, filecontent)
    # URL: http://Host/Index/Type
    os.system('curl -XPOST http://h-mstr:9200/documents/
attachment -d @'
            +TMP_FILE_NAME)
    os.remove(TMP_FILE_NAME)
```

If all goes well, your files will be stored in the HBase table and indexed in Elasticsearch.

Search and retrieve

You can search the index 'documents' for a keyword expression as follows:

```
#!/usr/bin/python
import sys
from elasticsearch import Elasticsearch
def search(es_index, query_string):
    es = Elasticsearch(['h-mstr'])
    query_body = {'query':
        {'query_string':
            {'query': query_string}},
        'fields':['title']}
    # response is a dictionary with nested dictionaries
    response = es.search(index=es_index, body=query_body)
    return [hit['fields']['title'] for hit in response['hits']]
['hits']]
# test run the module with search expression as a parameter
if __name__=='__main__':
    print search('documents', ' '.join(sys.argv[1:]))
```

The following example lets you get the document from the HBase documents table by supplying the key:

```
#!/usr/bin/python
import sys
import happybase
def fetch_attachment(table, full_filename):
    row = table.row(full_filename)
    filename = full_filename.split('/')[1]
    f = open(filename, 'wb')
    f.write(row['content':data'])
    f.close()
    return filename
# test run the module with the key (full filename) as
parameter
if __name__=='__main__':
    connection = happybase.Connection('h-mstr')
    table = connection.table('documents')
    print "Document File: ", fetch_attachment(table, sys.
argv[1])
```

The *search* and *fetch_attachment* methods can be combined to create versatile and very useful Web applications for any organisation that works with a very large number of documents. 

By: Anil Seth

The author has earned the right to do what interests him. You can find him online at <http://sethani.com>, <http://sethani.blogspot.com>, and reach him via email at anil@sethani.com